Week 11

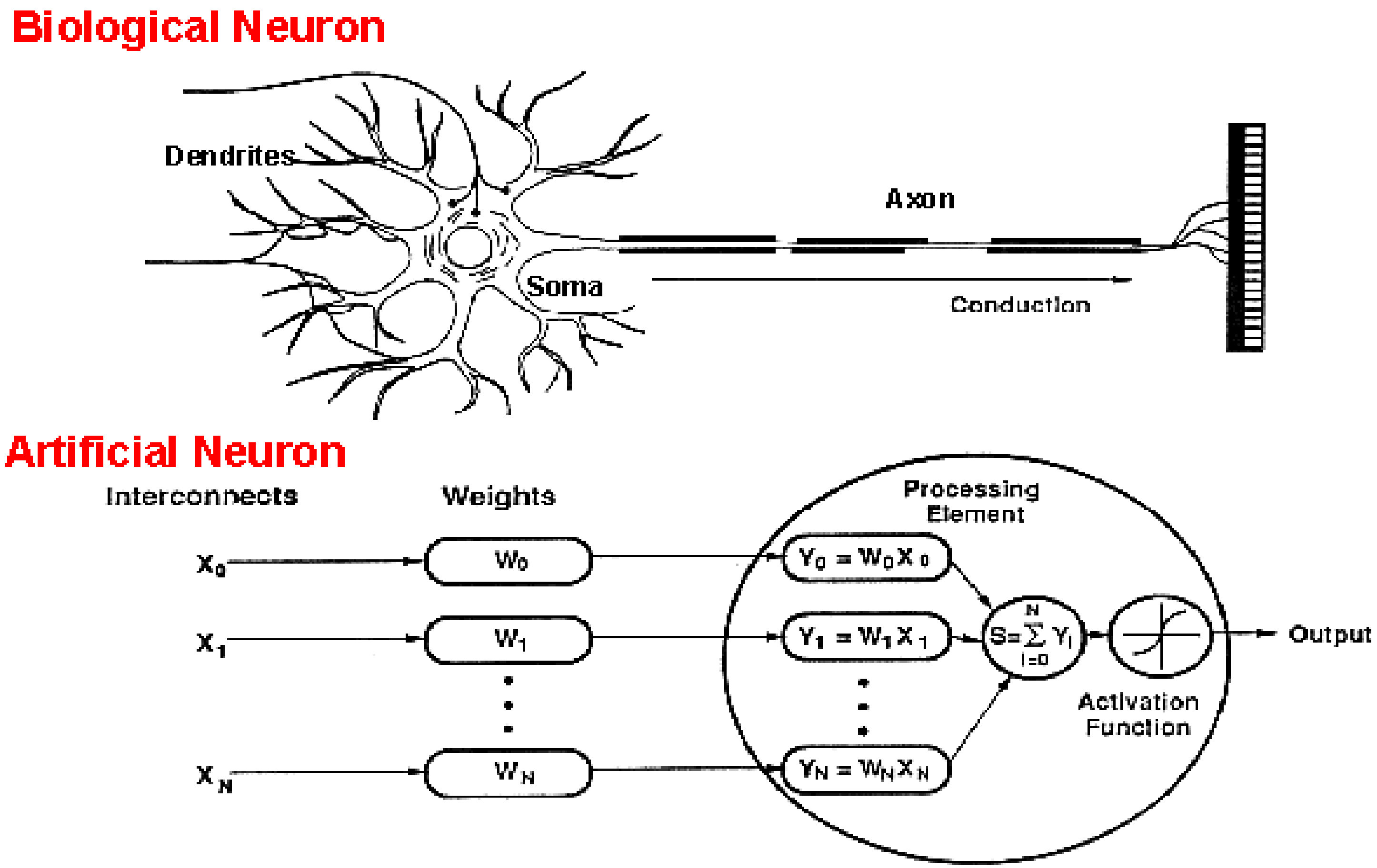
Artificial Neural Networks (ANN)

- **Basic Idea:** A complex non-linear function can be learned as a composition of simple processing units

- ANN is a collection of simple processing units (nodes) that are connected by directed links (edges)
  - Every node receives signals from incoming edges, performs computations, and transmits signals to outgoing edges
  - Analogous to *human brain* where nodes are neurons and signals are electrical impulses
  - Weight of an edge determines the strength of connection between the nodes

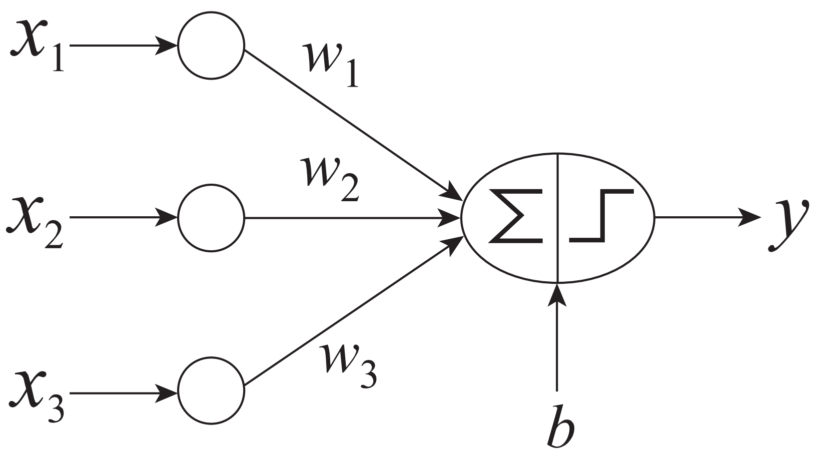
- Simplest ANN: **Perceptron** (single neuron)

How do ANNs work?



An artificial neuron is an imitation of a human neuron

Basic Architecture of Perceptron



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T\mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

$$\tilde{\mathbf{w}} = (\mathbf{w}^T \;\; b)^T \qquad \tilde{\mathbf{x}} = (\mathbf{x}^T \;\; 1)^T$$

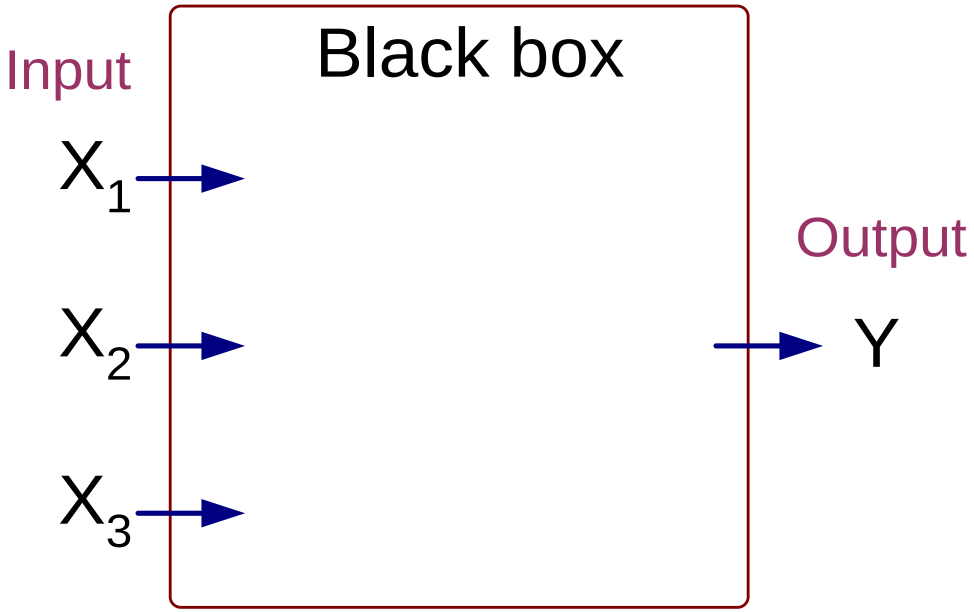
$$\hat{y} = sign(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}})$$

Activation Function

- Learns linear decision boundaries
- Related to logistic regression (activation function is sign instead of sigmoid)

Perceptron Example

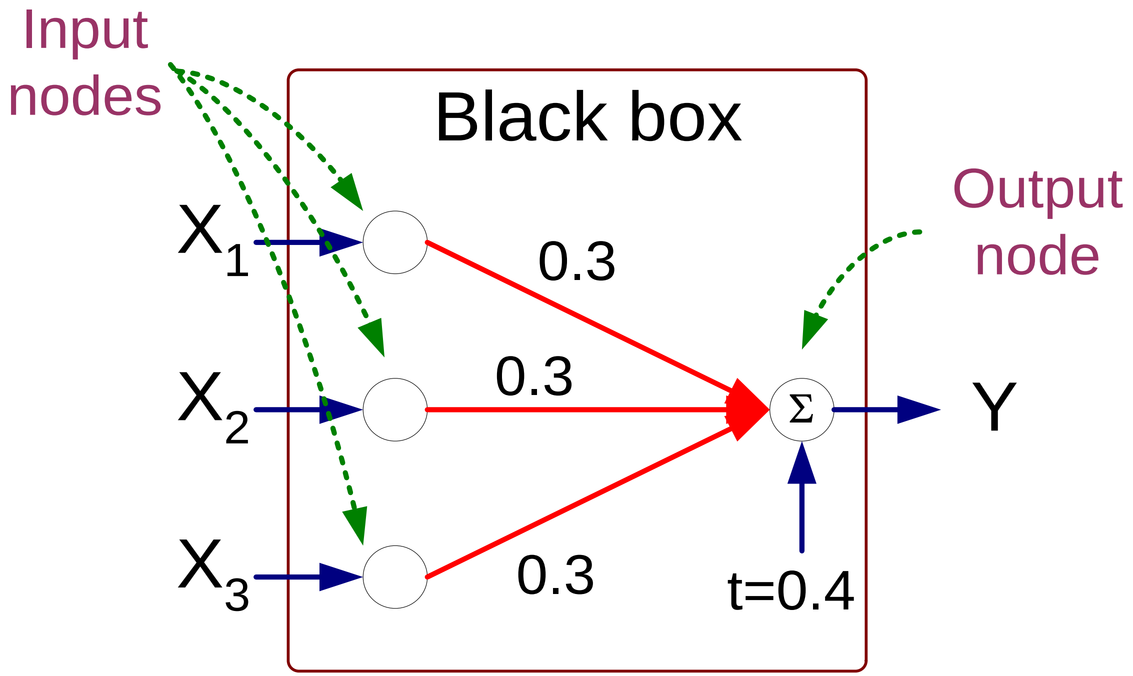
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



Output Y is 1 if at least two of the three inputs are equal to 1.

Perceptron Example

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



$$Y = sign(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } sign(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Perceptron Learning Rule

- Initialize the weights (w_0, w_1, ..., w_d)
- Repeat
  - For each training example (x_i, y_i)
    - Compute $\widehat{y_i}$
    - Update the weights:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda\big(y_i - \hat{y}_i^{(k)}\big)x_{ij}$$

- Until stopping condition is met

- k: iteration number; λ: learning rate

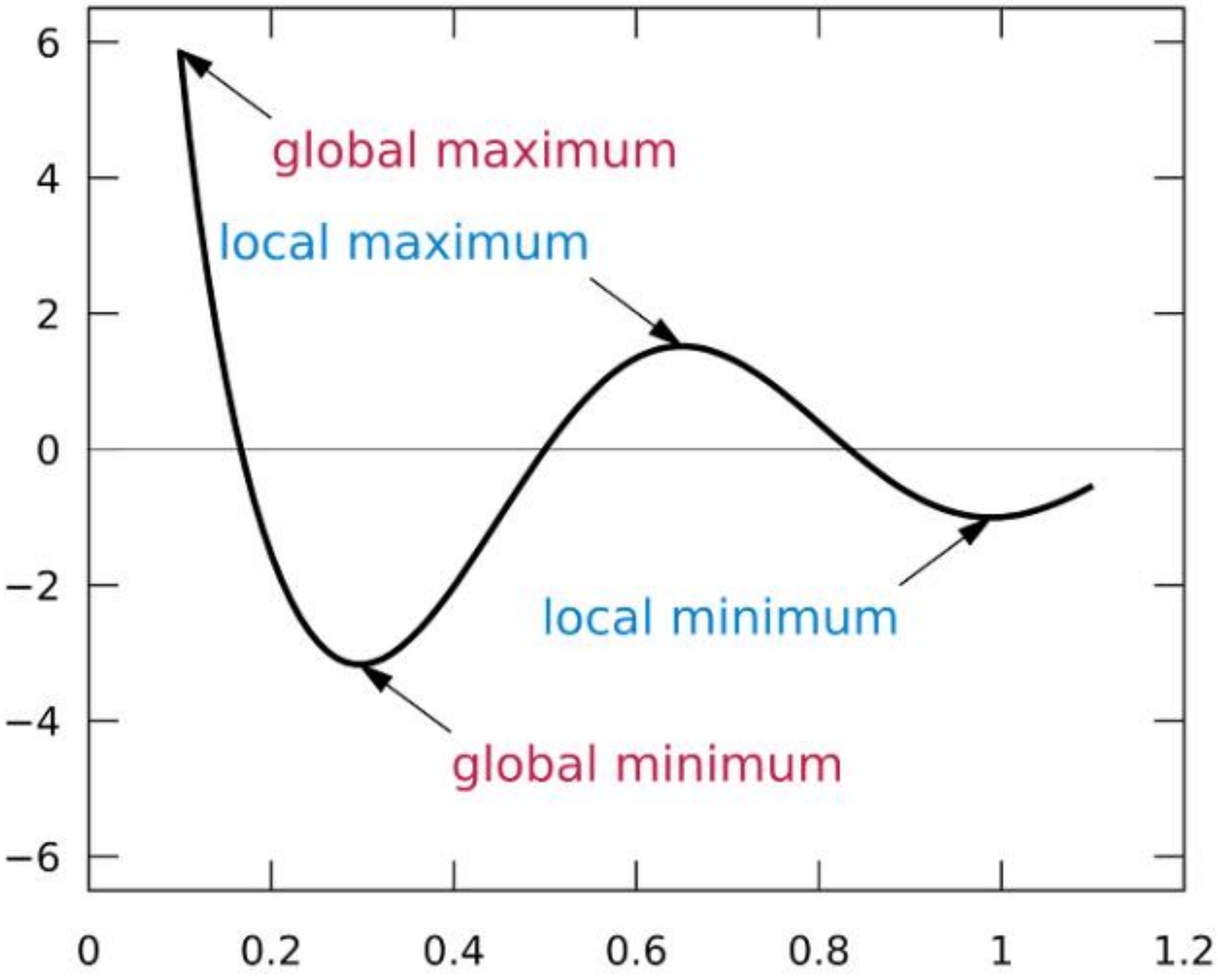
The **learning rate** is a key hyperparameter in neural networks that controls how much the model's weights are adjusted during training in response to the gradient of the loss function.

How It Affects Training:

1. **Small Learning Rate (η):**
   - **Effect**: Slow convergence because weights are updated in very small increments.
   - **Advantage**: More stable; less likely to overshoot the optimal solution.
   - **Disadvantage**: Can get stuck in local minima or take a long time to converge.
2. **Large Learning Rate (η):**
   - **Effect**: Faster weight updates and quicker convergence.
   - **Advantage**: Can escape shallow local minima.
   - **Disadvantage**: May overshoot the minimum or cause divergence (oscillations).
3. **Adaptive Learning Rate:**
   - Some optimization methods, like **Adam** or **RMSprop**, dynamically adjust the learning rate during training to balance stability and speed.

Local Minimum vs Global Minimum

- In machine learning, local minima and global minima are two important concepts related to the optimization of loss functions. A loss function is a function that measures the error between a model's predictions and the ground truth. The goal of machine learning is to find a model that minimizes the loss function.
- A local minimum is a point in the parameter space where the loss function is minimized in a local neighborhood. A global minimum is a point in the parameter space where the loss function is minimized globally.



Epochs: An **epoch** in the context of machine learning and CNNs refers to **one complete pass of the entire training dataset through the model**. During an epoch, the model processes all the training examples once, allowing it to learn patterns and adjust its weights accordingly.

How Epochs Work:

1.Training Dataset Splitting:

1. The dataset is divided into smaller batches (mini-batches).
2. Each batch is processed independently during an epoch to update the model's parameters.

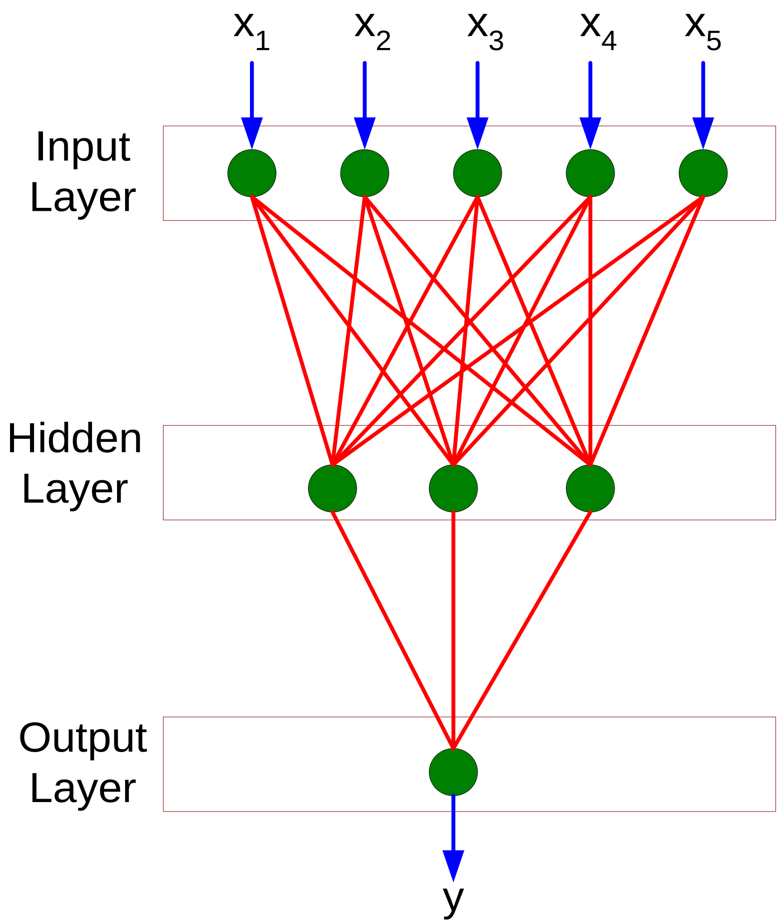
2.Completing an Epoch:

1. Once all batches of the dataset are processed, an epoch is considered complete.
2. The weights of the model have been updated multiple times (once per batch).

3.Repeating for Better Accuracy:

1. Multiple epochs are used to improve the model's performance.
2. The model learns and refines its parameters with each successive epoch.

Multi-layer Neural Network



- More than one *hidden layer* of computing nodes
- Every node in a hidden layer operates on activations from preceding layer and transmits activations forward to nodes of next layer
- Also referred to as “feedforward neural networks”

Difference Between Perceptron and Neural Network

1. Definition:

- **Perceptron**:
  A perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt in 1958. It models a single-layer linear classifier capable of binary classification.
- **Neural Network**:
  A neural network is a collection of interconnected layers of perceptrons (or artificial neurons) designed to solve more complex problems. It includes input, hidden, and output layers, allowing non-linear transformations.

2. Architecture:

- **Perceptron**:
  - Consists of a single layer with one or more inputs and a single output.
  - Processes inputs using a weighted sum, applies an activation function (e.g., step function), and outputs a single result.
- **Neural Network**:
  - Comprises multiple layers: an input layer, one or more hidden layers, and an output layer.
  - Employs various activation functions (ReLU, sigmoid, etc.) to capture non-linear patterns.

3. Complexity:

- **Perceptron**:
  - Only capable of solving **linearly separable** problems (e.g., AND, OR functions).
  - Cannot solve non-linear problems like XOR.
- **Neural Network**:
  - Can solve both **linear and non-linear** problems due to its multi-layer architecture and complex activation functions.
  - Suitable for tasks like image recognition, speech processing, and natural language understanding.

4. Learning Process:

- **Perceptron**:
  - Uses a simple learning rule to adjust weights: the Perceptron Learning Algorithm.
  - Stops learning once all samples are classified correctly.
- **Neural Network**:
  - Employs more sophisticated algorithms like **backpropagation** combined with optimization methods (e.g., stochastic gradient descent).
  - Continues learning over multiple iterations to minimize a loss function.

5. Use Cases:

- **Perceptron:**
  - Primarily used for educational purposes or as a simple introduction to machine learning concepts.
  - Example: Binary classification tasks with linear boundaries.
- **Neural Network:**
  - Widely applied in real-world scenarios requiring high complexity, such as image classification, language translation, and predictive analytics.

Summary Table:

Feature	Perceptron	Neural Network
Layers	Single-layer	Multi-layer (input, hidden, output)
Complexity	Solves linear problems	Solves linear and non-linear problems
Learning Rule	Perceptron Learning Algorithm	Backpropagation, optimization methods
Use Cases	Simple binary classification	Complex tasks like vision and NLP
Activation	Step function	Advanced functions (ReLU, sigmoid, etc.)

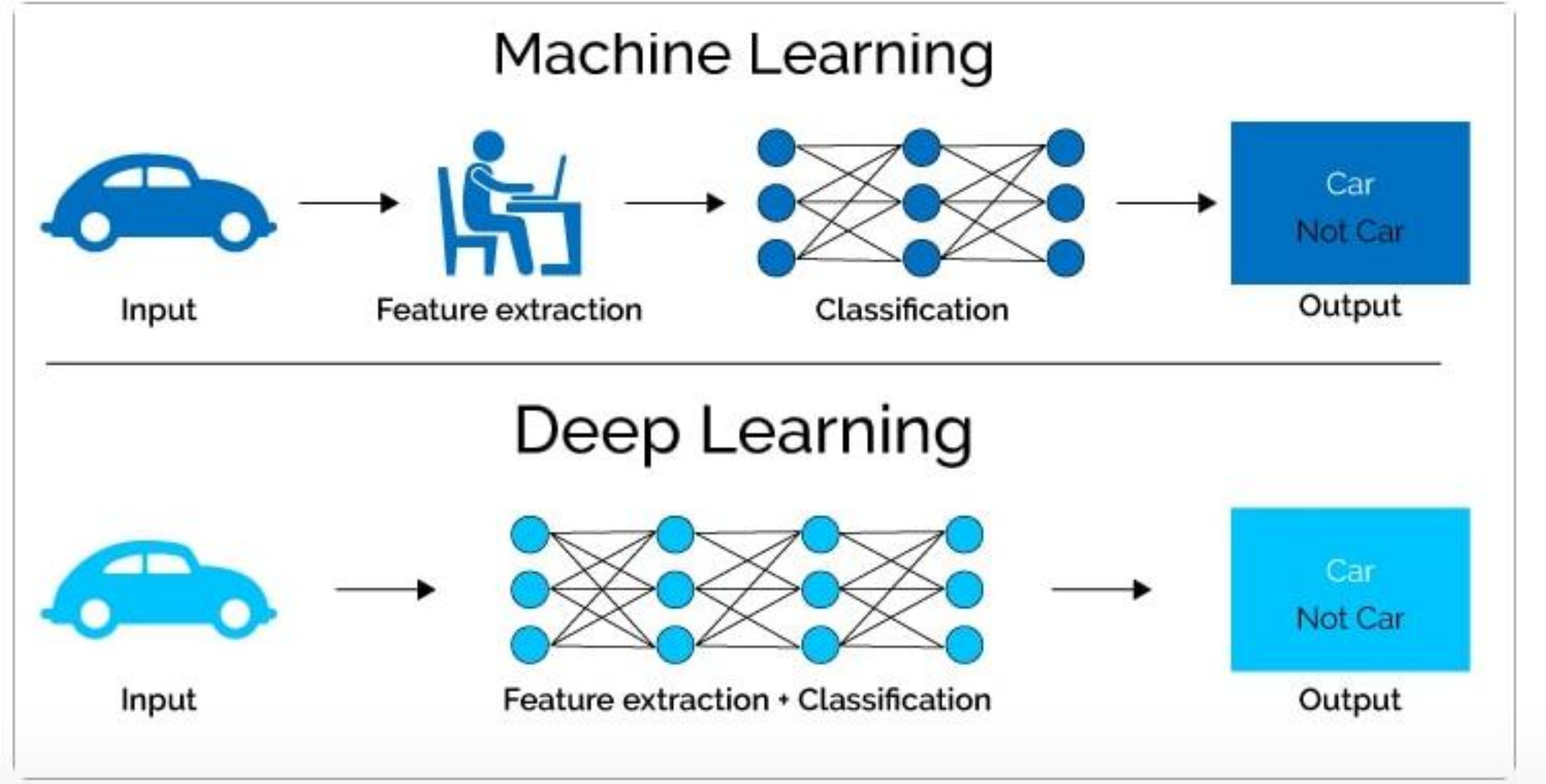
In essence, the perceptron is a foundational concept in machine learning, while neural networks build upon it to tackle more sophisticated and real-world problems.

Deep learning vs. machine learning

Machine Learning is the general term for when computers learn from data. It describes the intersection of computer science and statistics where algorithms are used to perform a specific task without being explicitly programmed; instead, they recognize patterns in the data and make predictions once new data arrives.

Deep Learning algorithms can be regarded both as a sophisticated and mathematically complex evolution of machine learning algorithms. The field has been getting lots of attention lately and for good reason: Recent developments have led to results that were not thought to be possible before.

Deep learning vs. machine learning



Deep learning vs. machine learning

Machine learning	Deep learning
A subset of AI	A subset of machine learning
Can train on smaller data sets	Requires large amounts of data
Requires more human intervention to correct and learn	Learns on its own from environment and past mistakes
Shorter training and lower accuracy	Longer training and higher accuracy
Makes simple, linear correlations	Makes non-linear, complex correlations
Can train on a CPU (central processing unit)	Needs a specialized GPU (graphics processing unit) to train

Computer vision is a field of artificial intelligence (AI) that uses machine learning and neural networks to teach computers and systems to derive meaningful information from digital images, videos and other visual inputs—and to make recommendations or take actions when they see defects or issues.

Computer vision tasks

Image classification (or image recognition): Image processing involves **performing operations on an image to make it better or to get important information from it**. It's like fixing or improving a picture, and it's a bit like working with signals. The input is an image, and the output can be a better image or some important details from the image. It aims to classify images according to defined categories. A rudimentary example of this is CAPTCHA image tests, in which a group of images may be organized as images with stop signs and images without. Image classification assigns one label to a whole image.

Object detection, by comparison, delineates individual objects in an image according to specified categories. While image classification divides images among those that have stop signs and those that do not, object detection locates and categorizes all of the road signs in an image, as well as other objects such as cars and people.